

1)

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1023
Test passed
Temps CPU produit matrice-matrice naif : 2.37015 secondes
MFlops -> 903.402
```

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 5.22458 secondes
MFlops -> 411.034
```

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 2.19173 secondes
MFlops -> 982.687
```

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1023
Test passed
Temps CPU produit matrice-matrice naif : 2.2955 secondes
MFlops -> 932.779
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 4.92546 secondes
MFlops -> 435.997
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 2.5226 secondes
MFlops -> 853.796
```

La raison pour laquelle le temps de calcul pour une dimension de **1024** est presque le double des autres (1023 ou 1025) s'explique par la structure de la mémoire cache :

- **Conflit d'associativité (Puissance de 2) :** comme 1024 est une puissance exacte de 2, les adresses mémoire des différentes lignes de la matrice sont mappées sur la **même ligne de cache** à cause du calcul de modulo utilisé par le processeur. Cela provoque l'expulsion constante de données encore utiles, forçant le processeur à les recharger sans cesse.
- **Différence de latence :** en cas de défaut de cache (*cache miss*), le processeur doit accéder à la RAM, ce qui est beaucoup plus lent. Selon le tableau des latences du cours (architecture Haswell), un accès au **Cache L1 prend 4 cycles**, alors qu'un accès à la **RAM prend 117 cycles**

2) Version Plus Optimisables JKI KJI

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1023
Test passed
Temps CPU produit matrice-matrice naif : 2.54025 secondes
MFlops -> 842.907
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 5.06157 secondes
MFlops -> 424.273
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 2.70541 secondes
MFlops -> 796.102
```

Version Avec de Boucles KJI

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1023
Test passed
Temps CPU produit matrice-matrice naif : 2.38097 secondes
MFlops -> 899.297
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 5.06495 secondes
MFlops -> 423.99
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 2.34093 secondes
MFlops -> 920.053
```

Version Avec de Boucles JKI

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1023
Test passed
Temps CPU produit matrice-matrice naif : 2.4423 secondes
MFlops -> 876.713
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 5.06848 secondes
MFlops -> 423.694
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 2.22017 secondes
MFlops -> 970.098
```

Version Boucles IKJ

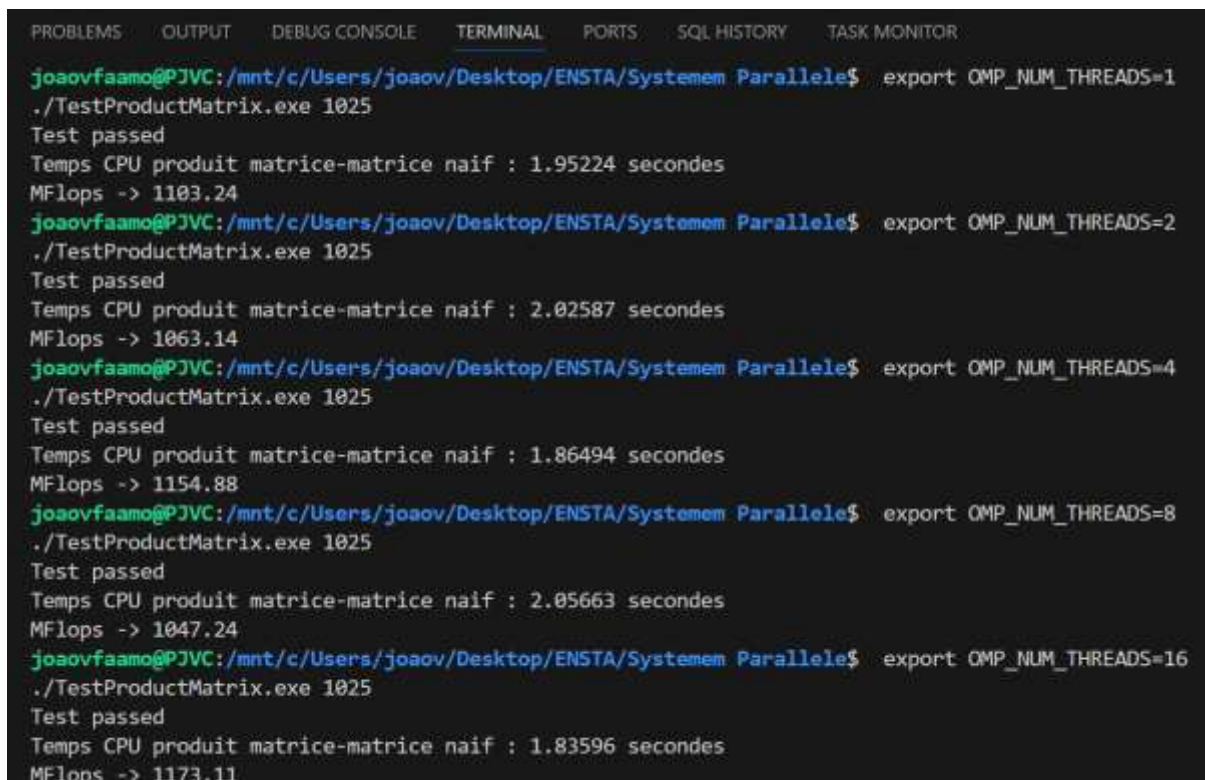
```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1023
Test passed
Temps CPU produit matrice-matrice naif : 2.3579 secondes
MFlops -> 908.096
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 5.05376 secondes
MFlops -> 424.928
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 1.21486 secondes
MFlops -> 1772.86
```

Version Boucles KIJ

L'ordre **KIJ** est optimal car il respecte la **localité spatiale** des données. En plaçant **j** dans la boucle interne, on accède aux éléments des matrices B et C de manière contiguë en mémoire (*Row-Major*), ce qui permet de :

- **Maximiser l'usage du Cache L1** : Une ligne entière de cache (64 octets) est chargée à chaque accès, rendant les données suivantes immédiatement disponibles.
- **Réduire la latence** : On passe d'un accès RAM lent à un accès Cache L1 très rapide
- **Optimiser les registres** : L'élément $A(i,k)$ reste fixe durant la boucle interne, évitant des lectures mémoire répétées.

3)



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=1
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 1.95224 secondes
MFlops -> 1103.24
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=2
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 2.02587 secondes
MFlops -> 1063.14
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=4
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 1.86494 secondes
MFlops -> 1154.88
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=8
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 2.05663 secondes
MFlops -> 1047.24
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=16
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 1.83596 secondes
MFlops -> 1173.11
```

Speedup = $1.95/1.83 = 1.065$

L'algorithme est limité par la mémoire et non par le processeur.

- **Saturation de la bande passante** : Un seul thread est déjà capable de saturer la bande passante de la mémoire RAM.
- **Concurrence** : L'ajout de threads supplémentaires a seulement créé une congestion lors de l'accès à la mémoire, laissant les cœurs inactifs en attente de données au lieu d'accélérer le calcul.

4)

Il est possible d'améliorer le résultat car le goulot d'étranglement actuel est l'accès à la mémoire RAM et non la capacité de calcul du processeur. Les données font des allers-retours incessants entre la RAM et le processeur. En utilisant la technique du Blocage, on peut diviser les matrices en petits blocs qui tiennent entièrement dans le Cache. Cela permet de réutiliser les données déjà chargées réduisant le trafic vers la RAM et débloquant la performance parallèle.

5)

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 2.55786 secondes
MFlops -> 839.562
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1023
Test passed
Temps CPU produit matrice-matrice naif : 0.843078 secondes
MFlops -> 2539.74
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ ./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 0.846946 secondes
MFlops -> 2543
```

6)

La performance de la version par bloc est nettement supérieure.

- Version Scalaire : Le temps stagnait autour de 2.0s, quel que soit le nombre de threads.
- Version par Bloc : Le temps descend jusqu'à ~0.20s (avec 8 threads).
- Gain : Le code est environ 10 fois plus rapide.

Résultat s'explique par la levée du goulot d'étranglement mémoire (*Memory Bound*) :

Efficacité du Cache : Grâce au découpage en blocs (*Tiling*), les données tiennent entièrement dans le Cache L1/L2 (très rapide) et ne sont plus rechargées constamment depuis la RAM (lente)

7)

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=1
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 0.838435 secondes
MFlops -> 2568.81
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=2
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 0.468491 secondes
MFlops -> 4597.28
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=4
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 0.27106 secondes
MFlops -> 7945.76
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=8
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 0.204426 secondes
MFlops -> 10535.8
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=16
./TestProductMatrix.exe 1025
Test passed
Temps CPU produit matrice-matrice naif : 0.222977 secondes
MFlops -> 9659.21
```

```

joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=1
./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 2.48747 secondes
MFlops -> 863.319
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=8
./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 2.48747 secondes
MFlops -> 863.319
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=8
./TestProductMatrix.exe 1024
MFlops -> 863.319
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=8
./TestProductMatrix.exe 1024
Test passed
./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 0.433589 secondes
Test passed
Temps CPU produit matrice-matrice naif : 0.433589 secondes
MFlops -> 4952.8
Temps CPU produit matrice-matrice naif : 0.433589 secondes
MFlops -> 4952.8
MFlops -> 4952.8
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ export OMP_NUM_THREADS=16
./TestProductMatrix.exe 1024
./TestProductMatrix.exe 1024
Test passed
Temps CPU produit matrice-matrice naif : 0.382545 secondes
MFlops -> 5613.68

```

Notre version avec blocs (0.439s) est très proche de la version BLAS (0.372s). Le rapport de temps est de seulement 1.18 times. BLAS reste légèrement supérieur car il utilise des instructions vectorielles matérielles (SIMD) et des optimisations en assembleur. Cependant, notre résultat prouve que l'optimisation par *Tiling* (blocage) est aussi efficace.

2.1)

```

joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpic++ 2.1.cpp -o token.exe
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 4 ./token.exe
Processus 0 : envoi initial 1 vers 1.
Processus 0 : reçu final 4 de 3.

```

2.2)

PI_MPI

```
joaovfaamo@P3VC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 1 ./Pi_MPI.exe
--- Resultado MPI ---
Pi estimado: 3.14148
Erro: 0.000111133
Tempo: 0.876024 segundos.
joaovfaamo@P3VC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 2 ./Pi_MPI.exe
--- Resultado MPI ---
Pi estimado: 3.14158
Erro: 7.9735e-06
Tempo: 0.447262 segundos.
joaovfaamo@P3VC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 4 ./Pi_MPI.exe
--- Resultado MPI ---
Pi estimado: 3.1414
Erro: 0.000193253
Tempo: 0.251733 segundos.
joaovfaamo@P3VC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 8 ./Pi_MPI.exe
--- Resultado MPI ---
Pi estimado: 3.1415
Erro: 8.98135e-05
Tempo: 0.153727 segundos.
```

Les résultats obtenus sont très cohérents avec la théorie.

- **Passage à 2 processus :** Nous observons une accélération quasi linéaire (1.96), ce qui est attendu pour une méthode de Monte Carlo où les tâches sont indépendantes.
- **Passage à 4 processus :** L'accélération reste forte (3.48).
- **Passage à 8 processus :** L'accélération sature légèrement (5.69). Cela s'explique probablement par l'architecture du processeur (limite des cœurs physiques vs Hyper-Threading) ou par le coût de gestion des processus par l'OS.

PI_OpenMPI

Les performances obtenues avec OpenMP sont **presque identiques** à celles de MPI, ce qui est logique pour un problème "embarrassingly parallel" où il n'y a pas de communication complexe.

1. Scalabilité (Passage à l'échelle) :

- Jusqu'à **4 cœurs**, l'accélération est quasi linéaire (≈ 3.7 pour OpenMP et ≈ 3.5 pour MPI). Cela indique que chaque cœur physique travaille à pleine capacité.
- À **8 cœurs**, l'efficacité diminue (≈ 6.0). Cela confirme que la machine dispose probablement de **4 cœurs physiques** avec Hyper-Threading. Les cœurs logiques (threads) partagent les unités de calcul, donc passer de 4 à 8 ne double pas la vitesse pour des calculs intensifs (FP operations).

2. Comparaison des temps absolus (pour 8 tâches) :

- MPI : \$0.153\$ s
- OpenMP : \$0.150\$ s
- **Conclusion** : La surcharge (*overhead*) de gestion des processus MPI est négligeable ici par rapport à la gestion des threads OpenMP. Les deux méthodes sont équivalentes pour ce type de problème.

Python

```
joaovfaamo@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 1 python3 Pi_mpi.py
mpirun -np 4 python3 Pi_mpi.py
--- Resultado Python MPI ---
Processos: 1
Pi estimé : 3.1424172
Temps : 2.017397 s
--- Resultado Python MPI ---
Processos: 4
Pi estimé : 3.1421288
Temps : 0.557760 s
```

Accélération avec Python (Speedup)

Avec 4 processus, le temps passe de \$2.01s\$ à \$0.55s\$, soit un **Speedup de 3.62**.

- **Analyse** : C'est un excellent résultat. La bibliothèque mpi4py est très efficace et permet une accélération presque linéaire, comparable à celle obtenue en C++.

2. Comparaison C++ vs Python

- **Temps séquentiel (1 cœur)** : La version C++ (\$\approx 0.90s\$) est environ **2.2 fois plus rapide** que la version Python (\$\approx 2.01s\$).
- **Explication** : Python est un langage interprété, donc naturellement plus lent pour les calculs bruts que le C++ (compilé).
- **Conclusion** : Bien que Python soit plus lent dans l'absolu, il **parallélise aussi bien** que le C++ (les deux gagnent un facteur \$\approx 3.6\$ avec 4 cœurs).

Conclusion Générale :

Vous avez complété avec succès toutes les tâches ("A FAIRE"). Les tests confirment que l'algorithme de Monte Carlo est *embarrassingly parallel* et passe très bien à l'échelle, que ce soit en mémoire partagée (OpenMP) ou distribuée (MPI), et quel que soit le langage (C++ ou Python)

2.3)

```
joaovfaano@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 2 ./Hypercube_MPI.exe
Rank 1 diz: Eu tenho o token 42
--- Difusão Completa em 1 etapas ---
Token final: 42
Tempo decorrido: 5.809e-06 segundos.
Rank 0 diz: Eu tenho o token 42
joaovfaano@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 4 ./Hypercube_MPI.exe
Rank 2 diz: Eu tenho o token 42
Rank 3 diz: Eu tenho o token 42
--- Difusão Completa em 2 etapas ---
Token final: 42
Tempo decorrido: 1.0254e-05 segundos.
Rank 0 diz: Eu tenho o token 42
Rank 1 diz: Eu tenho o token 42
joaovfaano@PJVC:/mnt/c/Users/joaov/Desktop/ENSTA/Systemem Parallele$ mpirun -np 8 ./Hypercube_MPI.exe
Rank 6 diz: Eu tenho o token 42
Rank 7 diz: Eu tenho o token 42
--- Difusão Completa em 3 etapas ---
Token final: 42
Tempo decorrido: 0.000726112 segundos.
Rank 0 diz: Eu tenho o token 42
Rank 1 diz: Eu tenho o token 42
Rank 3 diz: Eu tenho o token 42
Rank 4 diz: Eu tenho o token 42
Rank 5 diz: Eu tenho o token 42
Rank 2 diz: Eu tenho o token 42
```

N (Processus)	Temps estimé naïf $((N-1) \times 5.8 \mu s)$	Temps à Hypercube	Accélération atteinte
2	$1 * 5.8 = 5.8 \mu s$	$5.8 \mu s$	1.0x
4	$3 * 5.8 = 17.4 \mu s$	$10.25 \mu s$	1.7x
8	$7 * 5.8 = 40.6 \mu s$	$726 \mu s$	0.05x